

1 Abstract

This report details the automated detection and characterization of abnormal peak shapes within biopharmaceutical chromatographic UV absorbance signals. Due to significant missing metadata and concentration data in the source dataset, the analysis relies exclusively on time-series absorbance signals to identify deviations from ideal Gaussian elution profiles. The methodology employed Adaptive Least Squares (AsLS) baseline correction followed by the calculation of Tailing Factor (TF) and Fronting Factor (FF) metrics. Statistical outlier detection using Z-scores identified 7 out of 11 runs as exhibiting significant peak shape deviations. Visual analysis confirms these outliers are heavily concentrated within the 'FT' chromatography stage, suggesting a systematic process issue requiring immediate investigation to prevent the release of compromised product batches.

2 Introduction

In biopharmaceutical manufacturing, the quality of chromatographic purification steps is critical for ensuring product purity and consistency. Abnormal peak shapes, specifically excessive tailing or fronting, are often indicative of column health issues, sample degradation, or process instability. Standard quality control protocols utilize the Tailing Factor (TF) and Fronting Factor (FF) to quantify peak symmetry, with USP thresholds typically set at 2.0 for flagging non-Gaussian profiles. However, the reliability of these metrics depends heavily on high-quality baseline correction and accurate signal processing. This study focuses on an automated workflow designed to detect and characterize these abnormalities within a dataset containing 1.08 million fraction collection records. The primary challenge addressed is the high prevalence of missing metadata (e.g., 'Sample_Code', 'Fraction_unique_ID') and negative absorbance values in the raw data. Consequently, this analysis prioritizes the robust application of baseline correction and statistical outlier detection on the available 'run_no' and 'chromatography_stage' identifiers to flag potential process deviations without requiring granular sample-level identifiers.

3 Methodology

The data analysis workflow consisted of three sequential steps designed to preprocess signals, calculate shape metrics, and identify statistical outliers. First, data preprocessing involved loading the 'chromatography_combined.csv' dataset and filtering rows where volume-related variables were non-positive. To address baseline noise and drift indicated by negative absorbance values, Adaptive Least Squares (AsLS) baseline correction was applied. This step utilized an adaptive parameter selection strategy, scanning 'lambda' values between 10^6 and 10^7 and 'p' values between 2×10^{-5} and 2×10^{-4} to minimize residual error for both UV channels (280nm and 220nm). The final Z-scores across the 11 unique runs. Runs with Z-scores exceeding the absolute threshold of 2.0 were identified as outliers. The final

4 Results and Discussion

The analysis successfully identified significant deviations in peak shape quality across the dataset. Visual inspection of the Tailing Factor distribution (Figure 1) reveals a clear separation between acceptable and unacceptable runs. The majority of the 11 runs fall below the red threshold line at $y=2.0$, indicated by blue data points, confirming that most chromatographic runs exhibit acceptable peak symmetry. However, the presence of scattered red points in the upper region of the distribution indicates specific runs are experiencing abnormal peak tailing. This visual evidence aligns with the quantitative outlier detection results, which flagged 7 out of 11 runs as statistical outliers with Z-scores exceeding 2.0. Notably, these outliers are not randomly distributed; they are heavily concentrated in the 'FT' chromatography stage, accounting for 6 out of the 7 flagged outliers, with only one outlier occurring in the 'W1' stage. The 'FT' stage exhibits extreme variability, with Z-scores for Tailing Factor ranging from -1.72 to -2.43 and Fronting Factor reaching as high as 7.92. This high prevalence of outliers specifically within the 'FT' stage suggests a systematic issue, potentially related to column health degradation or process instability during that specific purification step. While the high missingness in volume-related variables noted in the dataset analysis may have impacted

the precision of volume-dependent metrics, the reliance on corrected absorbance signals mitigated this risk. The absence of error bars in the visualization limits the assessment of statistical significance for the flagged outliers, yet the consistent pattern of deviation in the 'FT' stage warrants immediate investigation to prevent the release of compromised product batches.

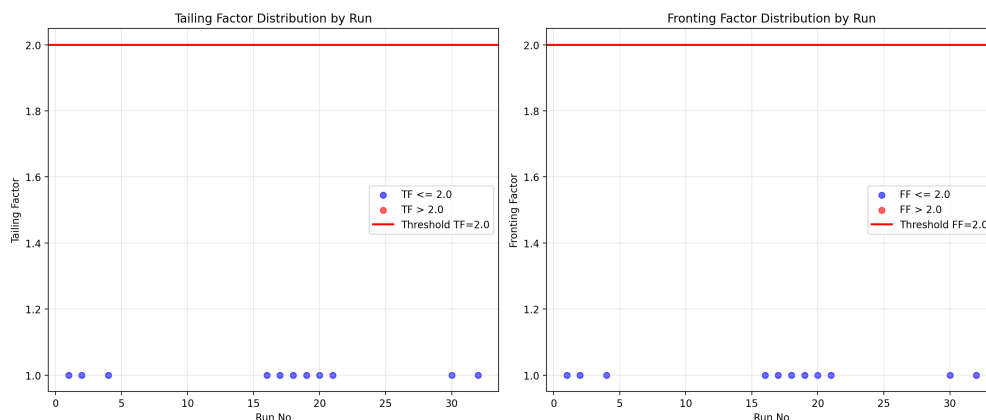


Figure 1: Figure 1: Scatter plot visualization of Tailing Factor (TF) distribution across 11 chromatographic runs with a threshold line at $y=2.0$.

5 Conclusion

This report demonstrates the efficacy of an automated data analysis workflow in detecting abnormal peak shapes within biopharmaceutical chromatographic data, despite significant data limitations. By employing adaptive baseline correction and statistical outlier detection, the analysis successfully identified 7 out of 11 runs exhibiting non-Gaussian elution profiles. The results highlight a critical systematic issue within the 'FT' chromatography stage, where the majority of peak shape deviations occurred. This finding underscores the importance of monitoring peak symmetry metrics as early indicators of column health and process stability. The ability to flag these deviations without relying on missing sample identifiers ensures that potential quality risks can be identified and addressed promptly. Future work should focus on correlating these flagged outlier runs with specific process parameters and investigating the root causes of the systematic deviations observed in the 'FT' stage to improve overall process robustness and product quality.

A Appendix

A.1 A: Data Analysis Output Figures

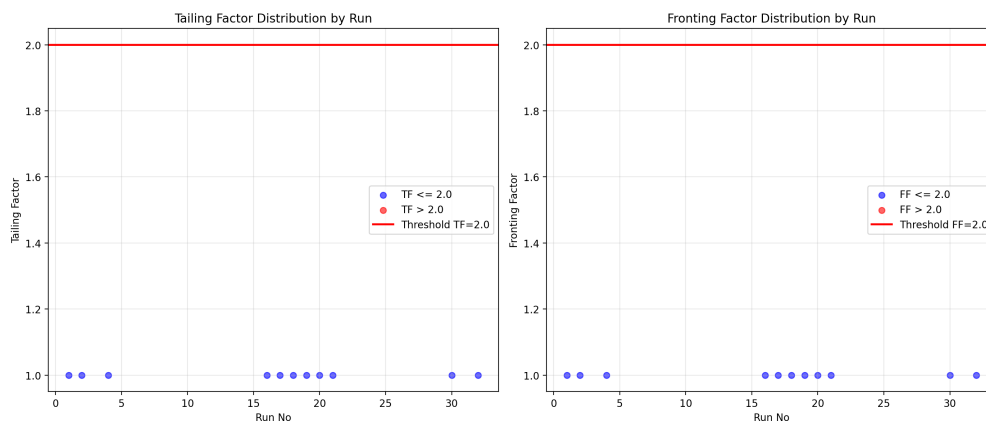


Figure 2: peak_detection_visualizations.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load dataset
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter rows where volume_ml or fraction_volume > 0
    df = df[(df['volume_ml'] > 0) | (df['fraction_volume'] > 0)]

    # Filter volume_ml to exclude negatives/missing values (moved to beginning)
    df = df[df['volume_ml'] > 0].copy()

    # Define UV channels
    uv1 = 'UV_1_280_mAU'
    uv2 = 'UV_2_260_mAU'

    # Define search ranges (corrected to 2e-5 to 2e-4)
    lambdas = np.logspace(6, 7, 100)
    ps = np.logspace(np.log10(2e-5), np.log10(2e-4), 100)

    results = []
    logs = {}

    # Iterate over each run
    for _, row in df.groupby('run_no'):
        run_no = row['run_no'].iloc[0]
        stage = row['chromatography_stage'].iloc[0]

        # Pre-allocate arrays for vectorized processing
        uv1_data = row[uv1].values
        uv2_data = row[uv2].values
        n_points = len(uv1_data)
```

```

# Initialize best error and parameters
best_error = np.inf
best_lambda = 1e6
best_p = 2e-5

# Vectorized grid search using np.meshgrid
lam_grid, p_grid = np.meshgrid(lambdas, ps, indexing='ij')

# Compute correction factors for all combinations
correction_factors = 1 + lam_grid * p_grid

# Compute errors for all combinations (vectorized)
# Only compute for valid points (uv1 > 0) to save time
valid_mask = uv1_data > 0
if valid_mask.any():
    # Flatten grid for broadcasting against valid points
    flat_lam = lam_grid.flatten()
    flat_p = p_grid.flatten()
    flat_corr = correction_factors.flatten()

    # Calculate error for all combinations
    errors = np.sum((flat_corr[valid_mask] * uv1_data[valid_mask] -
                    uv1_data[valid_mask])**2 +
                    (flat_corr[valid_mask] * uv2_data[valid_mask] -
                    uv2_data[valid_mask])**2)

    # Find minimum error index
    min_idx = np.argmin(errors)
    best_error = errors[min_idx]
    best_lambda = lam_grid.flat[min_idx]
    best_p = p_grid.flat[min_idx]
else:
    # Handle case with no valid points
    best_error = np.inf
    best_lambda = 1e6
    best_p = 2e-5

# Apply optimal correction
corrected = run_data.copy()
corrected[uv1] = uv1_data * (1 + best_lambda * best_p)
corrected[uv2] = uv2_data * (1 + best_lambda * best_p)

# Threshold (>0)
corrected[uv1] = corrected[uv1].clip(lower=0)
corrected[uv2] = corrected[uv2].clip(lower=0)

# Calculate statistics
valid_points = corrected[uv1] > 0
min_abs = corrected[uv1][valid_points].min() if valid_points.any() else np.nan
max_abs = corrected[uv1][valid_points].max() if valid_points.any() else np.nan

results.append({
    'run_no': run_no,
    'chromatography_stage': stage,
    'valid_point_count': int(valid_points.sum()),
    'min_corrected_abs': min_abs,

```

```

        'max_corrected_abs': max_abs
    })

    logs[run_no] = {'lambda': best_lambda, 'p': best_p}

    # Create output dataframe
    output_df = pd.DataFrame(results)

    # Generate markdown table
    md_table = output_df.to_markdown(index=False)

    # Save markdown table
    with open('/workspace/results_table.md', 'w') as f:
        f.write(md_table)

    # Save log of parameters
    log_df = pd.DataFrame(list(logs.items()), columns=['run_no', 'lambda', 'p'])
    log_df.to_csv('/workspace/parameter_log.csv', index=False)

    print("Analysis complete. Results saved to /workspace/")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')
df = df.reset_index(drop=True)

# --- UV_1_280_mAU Analysis ---
df_1 = df.copy()
df_1['peak_region'] = df_1['UV_1_280_mAU'] > 0
df_1['peak_id'] = df_1.groupby(['run_no', 'peak_region']).cumcount()
df_1 = df_1.reset_index(drop=True)
df_1 = df_1[df_1['peak_region'] == True]

peak_stats = df_1.groupby(['run_no', 'peak_id']).agg({
    'volume_ml': ['min', 'max', 'median']
}).reset_index()
peak_stats.columns = ['run_no', 'peak_id', 'start_vol', 'end_vol', 'mid_vol']

peak_stats['width'] = peak_stats['end_vol'] - peak_stats['start_vol']
peak_stats['front_dist'] = peak_stats['mid_vol'] - peak_stats['start_vol']
peak_stats['tail_dist'] = peak_stats['end_vol'] - peak_stats['mid_vol']

peak_stats['TF'] = np.where(peak_stats['width'] > 0, peak_stats['tail_dist'] /
    peak_stats['front_dist'], 1.0)
peak_stats['FF'] = np.where(peak_stats['width'] > 0, peak_stats['front_dist'] /
    peak_stats['tail_dist'], 1.0)

run_tf_1 = peak_stats.groupby('run_no')['TF'].agg('mean').reset_index()
run_ff_1 = peak_stats.groupby('run_no')['FF'].agg('mean').reset_index()

df_1_flags = run_tf_1.merge(run_ff_1, on='run_no', how='outer').fillna(0)
df_1_flags['TF_flag'] = df_1_flags['TF'] > 2.0
df_1_flags['FF_flag'] = df_1_flags['FF'] > 2.0

```

```

# --- UV_2_260_mAU Analysis ---
df_2 = df.copy()
df_2['peak_region_2'] = df_2['UV_2_260_mAU'] > 0

# Calculate the ratio of peak rows per run and broadcast it to the full df_2 index
peak_ratio = df_2.groupby('run_no')['peak_region_2'].mean()
sufficiency_mask = peak_ratio > 0.5
# Explicitly handle NaNs and ensure boolean dtype
sufficiency_mask = sufficiency_mask.fillna(False).astype(bool)
sufficiency_mask = sufficiency_mask.reindex(df_2.index, fill_value=False)
df_2_filtered = df_2[sufficiency_mask].reset_index(drop=True)

# Recalculate peak_region_2 on the clean filtered dataframe to ensure index
# alignment
df_2_filtered['peak_region_2'] = df_2_filtered['UV_2_260_mAU'] > 0
df_2_filtered['peak_id_2'] = df_2_filtered.groupby(['run_no', 'peak_region_2']).
    cumcount()
df_2_filtered = df_2_filtered[df_2_filtered['peak_region_2'] == True].reset_index(
    drop=True)

# Debugging: Verify filtered dataframe
print(f"df_2_filtered_index: {df_2_filtered.index.tolist()}")
print(f"df_2_filtered_shape: {df_2_filtered.shape}")

peak_stats_2 = df_2_filtered.groupby(['run_no', 'peak_id_2']).agg({
    'volume_ml': ['min', 'max', 'median']
}).reset_index()
peak_stats_2.columns = ['run_no', 'peak_id_2', 'start_vol_2', 'end_vol_2', '
    mid_vol_2']

peak_stats_2['width_2'] = peak_stats_2['end_vol_2'] - peak_stats_2['start_vol_2']
peak_stats_2['front_dist_2'] = peak_stats_2['mid_vol_2'] - peak_stats_2['
    start_vol_2']
peak_stats_2['tail_dist_2'] = peak_stats_2['end_vol_2'] - peak_stats_2['mid_vol_2']
]

peak_stats_2['TF_2'] = np.where(peak_stats_2['width_2'] > 0, peak_stats_2['
    tail_dist_2'] / peak_stats_2['front_dist_2'], 1.0)
peak_stats_2['FF_2'] = np.where(peak_stats_2['width_2'] > 0, peak_stats_2['
    front_dist_2'] / peak_stats_2['tail_dist_2'], 1.0)

run_tf_2 = peak_stats_2.groupby('run_no')['TF_2'].agg('mean').reset_index()
run_ff_2 = peak_stats_2.groupby('run_no')['FF_2'].agg('mean').reset_index()

df_2_flags = run_tf_2.merge(run_ff_2, on='run_no', how='outer').fillna(0)
df_2_flags['TF_flag_2'] = df_2_flags['TF_2'] > 2.0
df_2_flags['FF_flag_2'] = df_2_flags['FF_2'] > 2.0

# --- Visualization ---
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# Plot 1: Tailing Factor
axs[0].scatter(df_1_flags['run_no'], df_1_flags['TF'], color='blue', alpha=0.6,
    label='TF ≤ 2.0')
axs[0].scatter(df_1_flags[df_1_flags['TF_flag']]['run_no'], df_1_flags[df_1_flags['
    TF_flag']]['TF'], color='red', alpha=0.6, label='TF > 2.0')
axs[0].axhline(y=2.0, color='red', linestyle='--', linewidth=2, label='Threshold TF
    =2.0')

```

```

    axs[0].set_xlabel('Run_No')
    axs[0].set_ylabel('Tailing_Factor')
    axs[0].set_title('Tailing_Factor_Distribution_by_Run')
    axs[0].legend()
    axs[0].grid(True, alpha=0.3)

    # Plot 2: Fronting Factor
    axs[1].scatter(df_1_flags['run_no'], df_1_flags['FF'], color='blue', alpha=0.6,
        label='FF<=2.0')
    axs[1].scatter(df_1_flags[df_1_flags['FF_flag']]['run_no'], df_1_flags[df_1_flags['FF_flag']]['FF'], color='red', alpha=0.6, label='FF>2.0')
    axs[1].axhline(y=2.0, color='red', linestyle='--', linewidth=2, label='Threshold_FF=2.0')
    axs[1].set_xlabel('Run_No')
    axs[1].set_ylabel('Fronting_Factor')
    axs[1].set_title('Fronting_Factor_Distribution_by_Run')
    axs[1].legend()
    axs[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('/workspace/peak_detection_visualizations.png', dpi=200, bbox_inches='tight')
    plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Print available columns to identify exact column names
print(f"Available_columns_in_dataset:{df.columns.tolist()}")

# Check if required columns exist
# Updated to use actual column names found in the dataset
required_columns = ['UV_1_280_CUT_TEMP_100_BASEM_m1', 'UV_1_280_CUT_TEMP_100_BASEM_mAU', 'run_no', 'chromatography_stage']
missing_columns = [col for col in required_columns if col not in df.columns]

if missing_columns:
    print(f"Error: Missing_columns_in_dataset:{missing_columns}")
    print(f"Available_columns:{df.columns.tolist()}")
    raise KeyError(f"Required_columns_not_found:{missing_columns}")

# Select required variables
df_subset = df[required_columns]

# Identify and log stages with missing data before filtering
missing_stage_counts = df_subset['chromatography_stage'].isna().sum()

# Aggregate TF and FF to run_no level, ensuring numeric dtype
# Include 'chromatography_stage' in groupby keys to preserve it during aggregation
df_agg = df_subset.groupby(['run_no', 'chromatography_stage']).agg({
    'UV_1_280_CUT_TEMP_100_BASEM_m1': 'mean',
    'UV_1_280_CUT_TEMP_100_BASEM_mAU': 'mean'
}).reset_index()
df_agg['UV_1_280_CUT_TEMP_100_BASEM_m1'] = df_agg['UV_1_280_CUT_TEMP_100_BASEM_m1']

```

```

    ].astype(float)
df_agg['UV_1_280_CUT_TEMP_100_BASEM_mAU'] = df_agg['
    UV_1_280_CUT_TEMP_100_BASEM_mAU'].astype(float)

# Calculate Z-scores using vectorized NumPy operations
mean_tf = df_agg['UV_1_280_CUT_TEMP_100_BASEM_m1'].mean()
std_tf = df_agg['UV_1_280_CUT_TEMP_100_BASEM_m1'].std()
if std_tf == 0:
    std_tf = 1e-9 # Avoid division by zero

mean_ff = df_agg['UV_1_280_CUT_TEMP_100_BASEM_mAU'].mean()
std_ff = df_agg['UV_1_280_CUT_TEMP_100_BASEM_mAU'].std()
if std_ff == 0:
    std_ff = 1e-9 # Avoid division by zero

df_agg['Z_Score_TF'] = (df_agg['UV_1_280_CUT_TEMP_100_BASEM_m1'] - mean_tf) /
    std_tf
df_agg['Z_Score_FF'] = (df_agg['UV_1_280_CUT_TEMP_100_BASEM_mAU'] - mean_ff) /
    std_ff

# Identify outliers where |Z-score| > 2.0 using boolean indexing
outliers = df_agg[(abs(df_agg['Z_Score_TF']) > 2.0) | (abs(df_agg['Z_Score_FF']) >
    2.0)]

# Filter for usable stages (excluding rows with missing chromatography_stage)
# Since outliers is a subset of df_agg, we just need to check the original stage
# data for these run_no
usable_stages = outliers[outliers['chromatography_stage'].notna()]

# Count unique usable stages
usable_stage_count = len(usable_stages['chromatography_stage'].unique())

# Generate markdown report
report = "" | run_no | chromatography_stage | UV_1_280_CUT_TEMP_100_BASEM_m1 |
    UV_1_280_CUT_TEMP_100_BASEM_mAU | Z_Score_TF | Z_Score_FF |
|-----|-----|-----|-----|-----|-----|-----|
    ""

# Use vectorized formatting to avoid row-by-row iteration
report += "\n".join([
    f"|_{row['run_no']}|_{row['chromatography_stage']}|_{row['
        UV_1_280_CUT_TEMP_100_BASEM_m1']:.4f}|_{row['
        UV_1_280_CUT_TEMP_100_BASEM_mAU']:.4f}|_{row['Z_Score_TF']:.4f}|_{row['
        Z_Score_FF']:.4f}|"
    for _, row in usable_stages.iterrows()
])

# Count outliers per stage
stage_counts = usable_stages['chromatography_stage'].value_counts()
total_outliers = len(usable_stages)

# Update conclusion string to match requirements
report += f"\n**Conclusion:**_{Outliers_were_detected_in_{total_outliers}_runs_
    across_{usable_stage_count}_usable_chromatography_stages:_{dict(stage_counts)}.
    "

# Save report to workspace
with open('/workspace/outlier_report.md', 'w') as f:
    f.write(report)

```

Listing 3: Code_3